

PROGRAMACIÓN AVANZADA

APPLETS



Ing. Matias Macrino

APPLETS

Conceptos Básicos:

- ✓ **Applets:** son pequeños programas que se ejecutan dentro de una página Web, los cuales se pueden crear para diseños Web dinámicos e interactivos (similar a flash).
- ✓ **Los applets** se ejecutan desde un visualizador WWW (World Wide Web) – Browser –.
- ✓ Una referencia a un applet se incluye en una página Web al emplear una etiqueta HTML especial.
- ✓ No contienen el método **main()**.

APPLETS

- Para crear un **applet** se debe definir una subclase de la clase **Applet**, la cual está incluida en la librería java.applet.

```
import java.applet.*  
public class myClass extends Applet {  
    ...  
}
```

- La clase **Applet** ofrece:
 - el funcionamiento dentro del browser
 - incluir los elementos UI (User Interface)

APPLETS

▣ **Los applets** tienen diferentes tareas, relacionadas con eventos y métodos asociados, sobrescribiendo el método apropiado en cada subclase:

- ✓ La inicialización
- ✓ La pintura de pantalla
- ✓ Eventos del mouse.

▣ Métodos más relevantes a reescribir:

- ✓ Inicialización
- ✓ Arranque
- ✓ Detención
- ✓ Destrucción
- ✓ Pintura

APPLETS

Inicialización de applets:

- ▣ **Inicialización:** al cargar por primera vez el applet en el visualizador.
- ▣ Incluye la creación de los objetos que necesita, configuración de un estado inicial, cargado de imágenes o tipo y tamaño de letra y/o establecimiento de determinados parámetros.
- ▣ Para proporcionar un comportamiento para **la inicialización del applet**, se debe sobrescribir el **método `init()`**:

```
public void init() {  
    ...  
}
```

APPLETS

Arranque de applets:

□ Después de que se inicializa ***un applet***, lo siguiente es el arranque, lo cual ocurre si fue detenido previamente.

□ Para proporcionar el comportamiento de arranque, se debe sobrescribir el ***método start()***:

```
public void start() {  
  
    ...  
  
}
```

□ Incluir en este método, el arranque de un hilo para controlar el applet, enviar mensajes a los objetos auxiliares y también el arranque propiamente dicho del applet.

APPLETS

Detención de applets:

- La detención ocurre cuando el lector abandona la página que contiene un applet en ejecución, llamando al **método `stop()`**.
- Por defecto, al salir de la página Web, cualquier hilo que el applet haya iniciado continuará su ejecución.
- Al sobrescribir el **método `stop()`**, se puede suspender la ejecución de esos hilos y volverlos a arrancar si el applet se visualiza nuevamente:

```
public void stop() {  
    ...  
}
```

APPLETS

Destrucción de applets:

□ La destrucción le permite al applet limpiarse justo antes de que sea liberado, o de que el visualizador termine.

□ Para liberar un applet debe sobrescribirse el **método `destroy()`**.

```
public void destroy() {  
    ...  
}
```

□ El método **`destroy()`** cumple con la misma finalidad que el método `finalize()`.

APPLETS

Grafico o dibujo de applets:

- El dibujo es la forma en que un **applet dibuja algo en la pantalla**, ya sea un texto, una línea o una imagen.
- Esto se produce después de inicializarse o al redimensionar el browser.
- Para que el applet tenga una apariencia real en la pantalla, se debe sobrescribir el **método paint()**, que tiene como argumento un objeto de **Graphics**.
- Ese objeto lo crea el browser y es pasado al applet como parámetro. Se debe importar la librería AWT para el manejo de los gráficos.

```
public void paint(Graphics g) { . . . }
```

APPLETS

Ejemplo de applets sencillo:

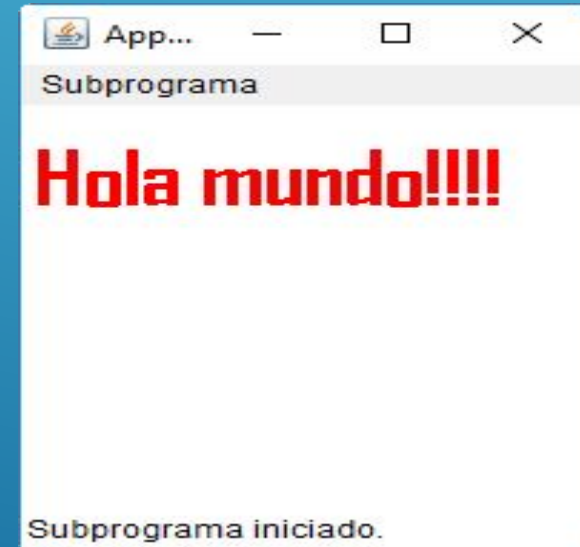
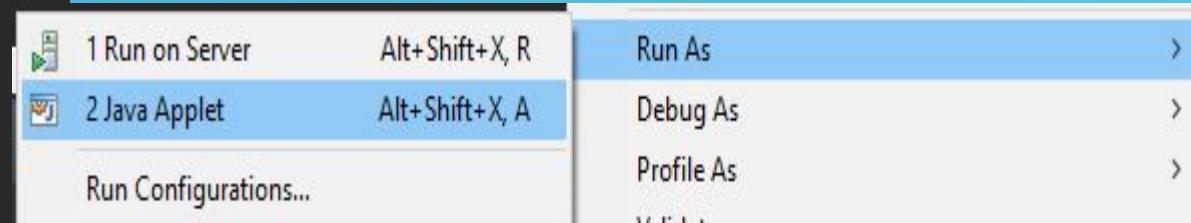
```
import java.applet.Applet;
import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics;

public class MiApplets extends Applet{

    Font fuente = new Font("Agency FB", Font.BOLD, 36);

    @Override
    public void paint(Graphics g) {
        g.setFont(fuente);
        g.setColor(Color.RED);
        g.drawString("Hola mundo!!!!", 5, 50);
    }

}
```



APPLETS

Ejemplo de applets sencillo:

- Para agregarlo a una **página Web**, se debe codificar la misma e incorporarlo. Existe una etiqueta especial **en HTML** para la ejecución de applets **<APPLET>**

Código HTML:

```
<HTML>
<HEAD>
  <TITLE>Esta página Web contiene un applet </TITLE>
</HEAD>
<BODY>
  <P>
  <BR>
  <APPLET CODE="MiApplets.class" WIDTH=200 HEIGHT=70>
    Deberia mostrar un applet en lugar de esto
  </APPLET>
</BODY>
</HTML>
```

APPLETS

- El atributo **CODE** indica el nombre del archivo de clase que contiene al applet.
- El archivo debe encontrarse en el mismo directorio que el **código HTML**. De lo contrario, agregar el atributo **CODEBASE** indicando el directorio

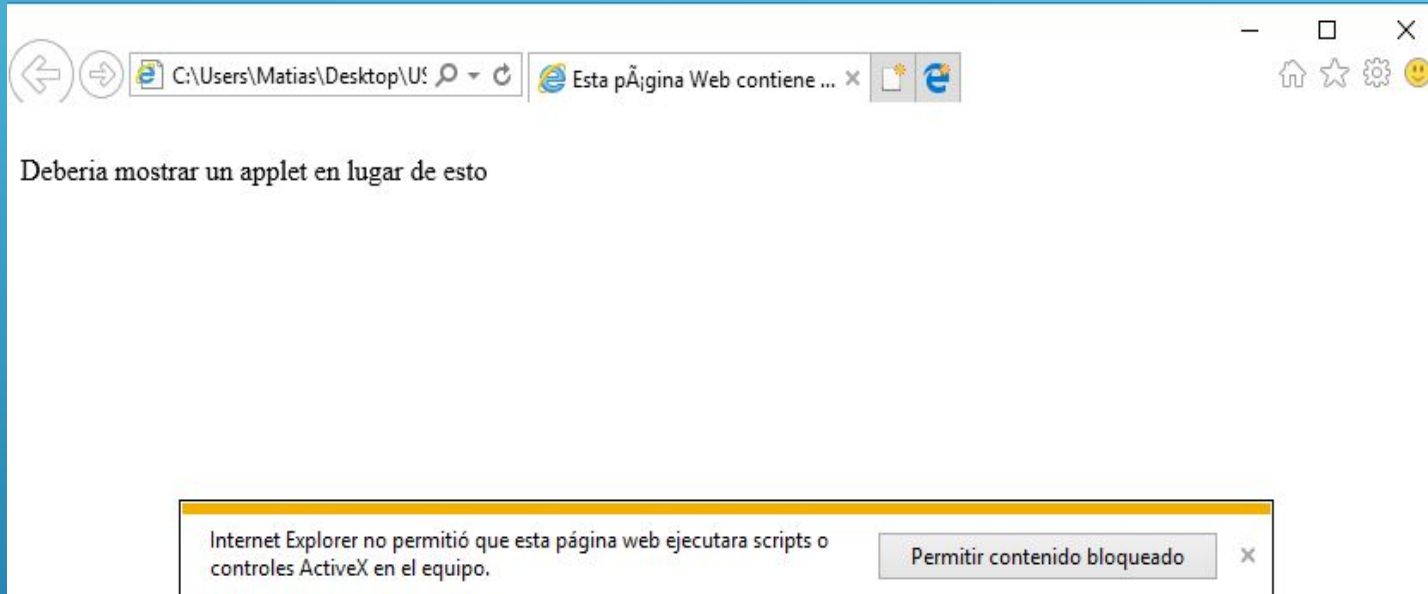
- Mismo directorio.

```
<APPLET CODE="MiApplets.class" WIDTH=200 HEIGHT=70>  
  Deberia mostrar un applet en lugar de esto  
</APPLET>  
BODY>
```

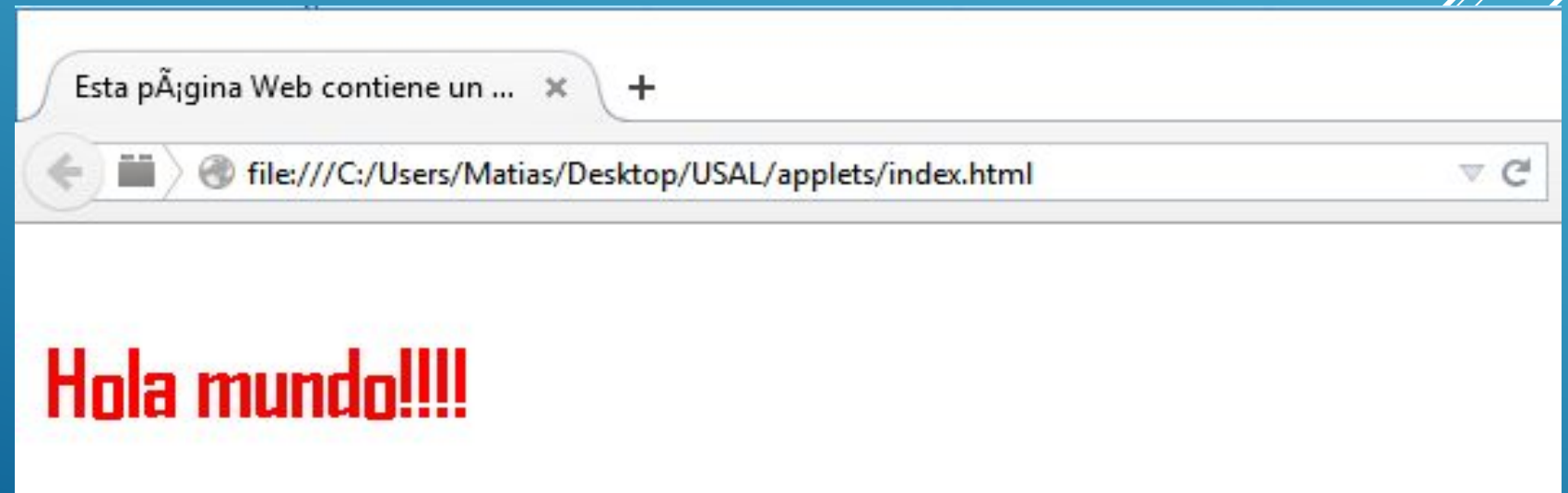
- En otro directorio.

```
<APPLET CODE="MiApplets.class" CODEBASE="C:\TEMP" WIDTH=200 HEIGHT=70>  
  Deberia mostrar un applet en lugar de esto  
</APPLET>  
BODY>
```

APPLETS



El texto entre las etiquetas **<APPLET>** y **</APPLET>** será mostrado en el caso que los browser no tengan la capacidad de ejecutar applets.



APPLETS

Parámetros a los applets:

- Como los **applets** no cuentan con una línea de comandos, se necesitan dos elementos para el pase de parámetros:
 - ❖ Una etiqueta de parámetros especial en el archivo **HTML**
 - ❖ Código en el **applet** para el tratamiento de los parámetros.
- Los parámetros de un **applet** incluyen dos partes: **un nombre cualquiera y el valor del parámetros en particular.**
- La etiqueta HTML para estos casos es **<PARAM>**, la cual tiene dos atributos: **NAME** y **VALUE**. Esta etiqueta **<PARAM>**, **debe estar informada dentro de las etiquetas <APPLET>** para abrir y cerrar.

APPLETS

Parámetros a los applets:

- El **método *init()*** es el encargado de capturar esos parámetros, mediante el método ***getParameter()***.
- El parámetro del método `getParameter()` corresponde con el nombre del argumento indicado en la etiqueta `<PARAM>`.
Si un parámetro esperado no es especificado en el archivo HTML, `getParameter()` devuelve “null”.

APPLETS

Ejemplo parámetros a los applets:

Se agrega el **método `init()`** para la captura de o los parámetros.

```
public class MiApplets extends Applet{

    Font fuente = new Font("Agency FB", Font.BOLD, 36);
    String nombre;

    @Override
    public void init() {
        this.nombre = getParameter("pName");
        if(this.nombre == null) {
            this.nombre = "Paramtero nulo";
        }
        this.nombre = "Hola " + this.nombre + "!!!!!!";
    }

    @Override
    public void paint(Graphics g) {
        g.setFont(fuente);
        g.setColor(Color.RED);
        g.drawString(this.nombre, 5, 50);
    }

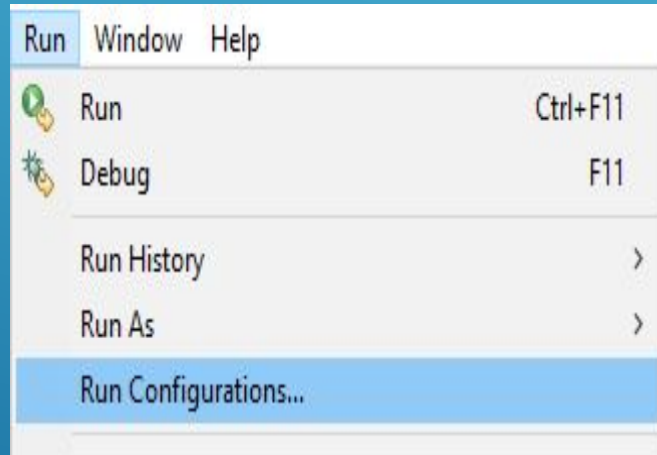
}
```

APPLETS

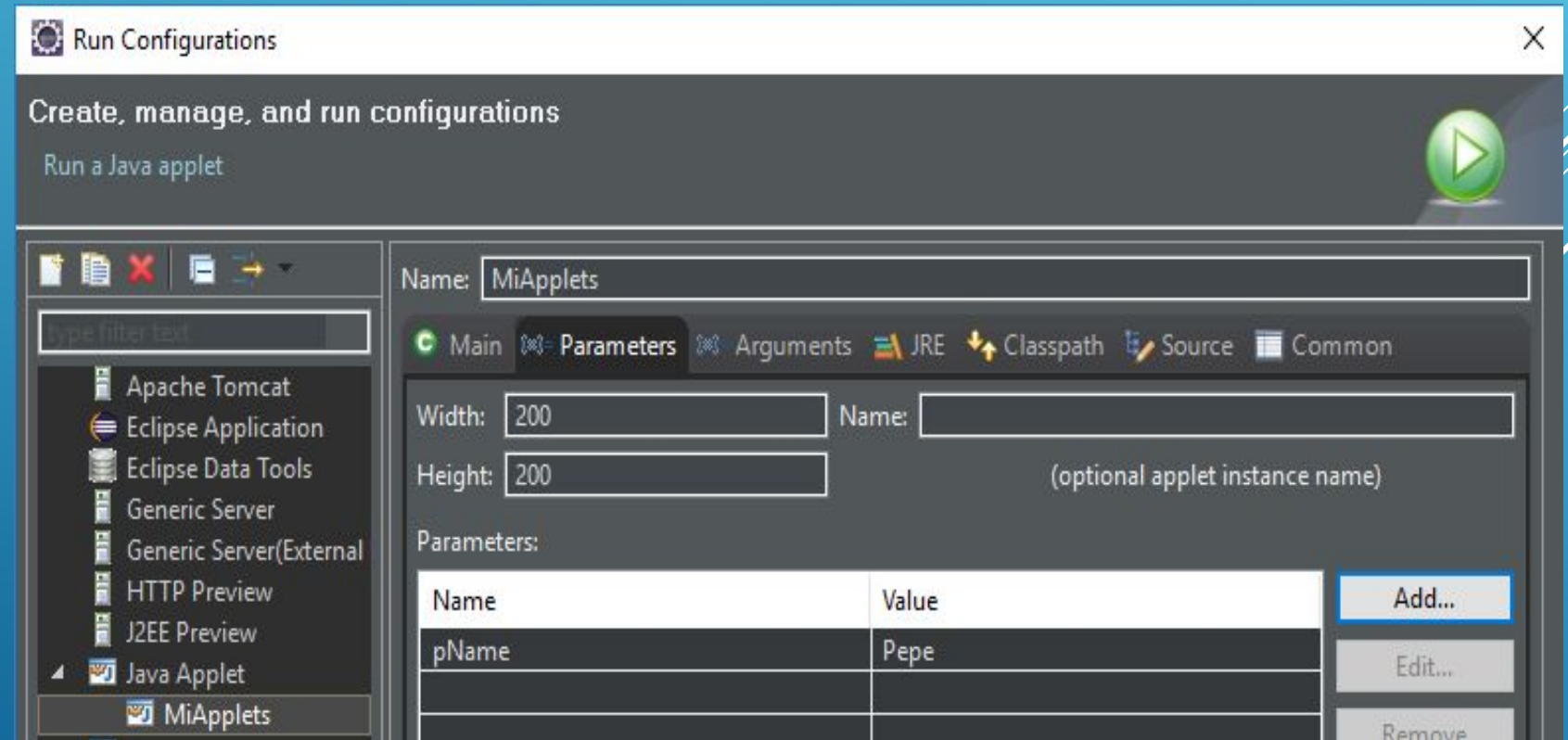
Ejemplo parámetros a los applets:

Setear parámetros desde eclipse.

Pestaña “Parameters” – “Add”



Menu “Run” ->
“Run Configuratuons”

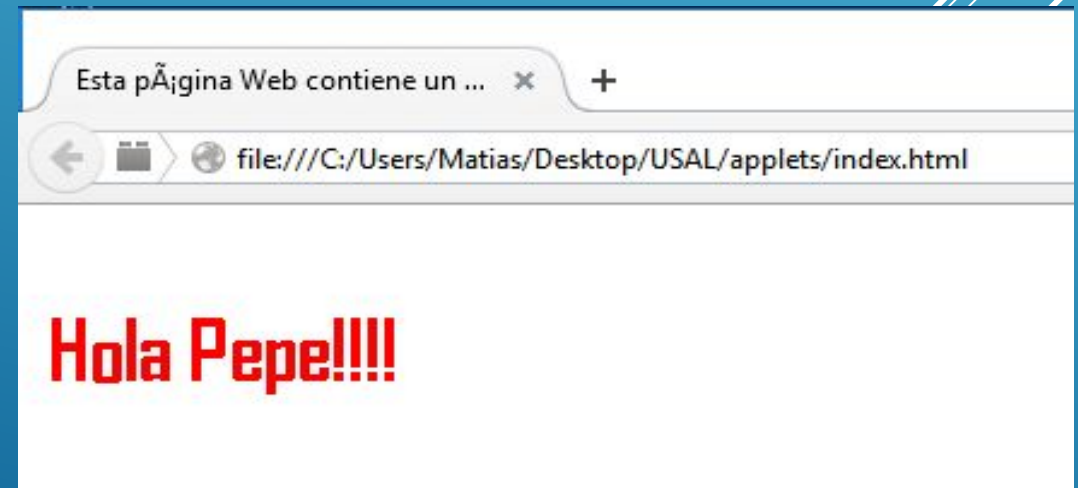


APPLETS

Ejemplo parámetros a los applets:

Desde código HTML.

```
<HTML>
<HEAD>
  <TITLE>Esta página Web contiene un applet </TITLE>
</HEAD>
<BODY>
  <P>
    <BR>
    <APPLET CODE="MiApplets.class" WIDTH=200 HEIGHT=70>
      <PARAM NAME="pName" VALUE="Pepe" />
      debería mostrar un applet en lugar de esto
    </APPLET>
  </BODY>
</HTML>
```



PROGRAMACIÓN AVANZADA

SERVLETS



Ing. Matias Macrino

SERVLET

Conceptos Básicos:

- ✓ **Servlets:** es un programa ejecutado en el servidor. Es un mecanismo para implantar aplicaciones en el lado del servidor, escritas en lenguaje JAVA que interactúa con clientes mediante el uso de un servidor a través del protocolo HTTP.
- ✓ Reciben peticiones y mandan resultados en HTTP, siendo el formato más común de salida una página HTML o un archivo XML (pero puede ser cualquier tipo MIME, una imagen, etc).
- ✓ Para programar servlets, los paquetes necesarios son:
 - ▣ **`javax.servlet`**
 - ▣ **`javax.servlet.http`**

SERVLET

Conceptos Básicos - Primer Servlet:

- ✓ Para generar un servlet debemos de extender de la clase **HttpServlet** y sustituir sus métodos **doGet** y **doPost**.
- ✓ Ambos métodos toman un par de argumentos: **HttpServletRequest** contiene métodos por los cuales puede encontrar información entrante como los datos del formulario, encabezado de petición HTTP y el nombre del host del cliente.
HttpServletResponse permite especificar la información de salida como los códigos de estado HTTP (200, 404, etc), encabezado de respuesta (Content-type, Set-cookie, etc) y lo mas importante permite obtener un **PrintWriter** utilizado para enviar el contenido del documento de regreso al cliente.

SERVLET

Proceso de ejecución de un Servlet:

- El proceso comienza con la petición HTTP que llega en primer lugar al servidor web.
- La petición se traslada al Contenedor de aplicaciones, en concreto a su motor del servicio Servlet/JSP (con su propia JVM).
- El motor encapsula la información de la petición en un objeto del tipo `HttpServletRequest`, además encapsula en un objeto `HttpServletResponse` el flujo de respuesta.

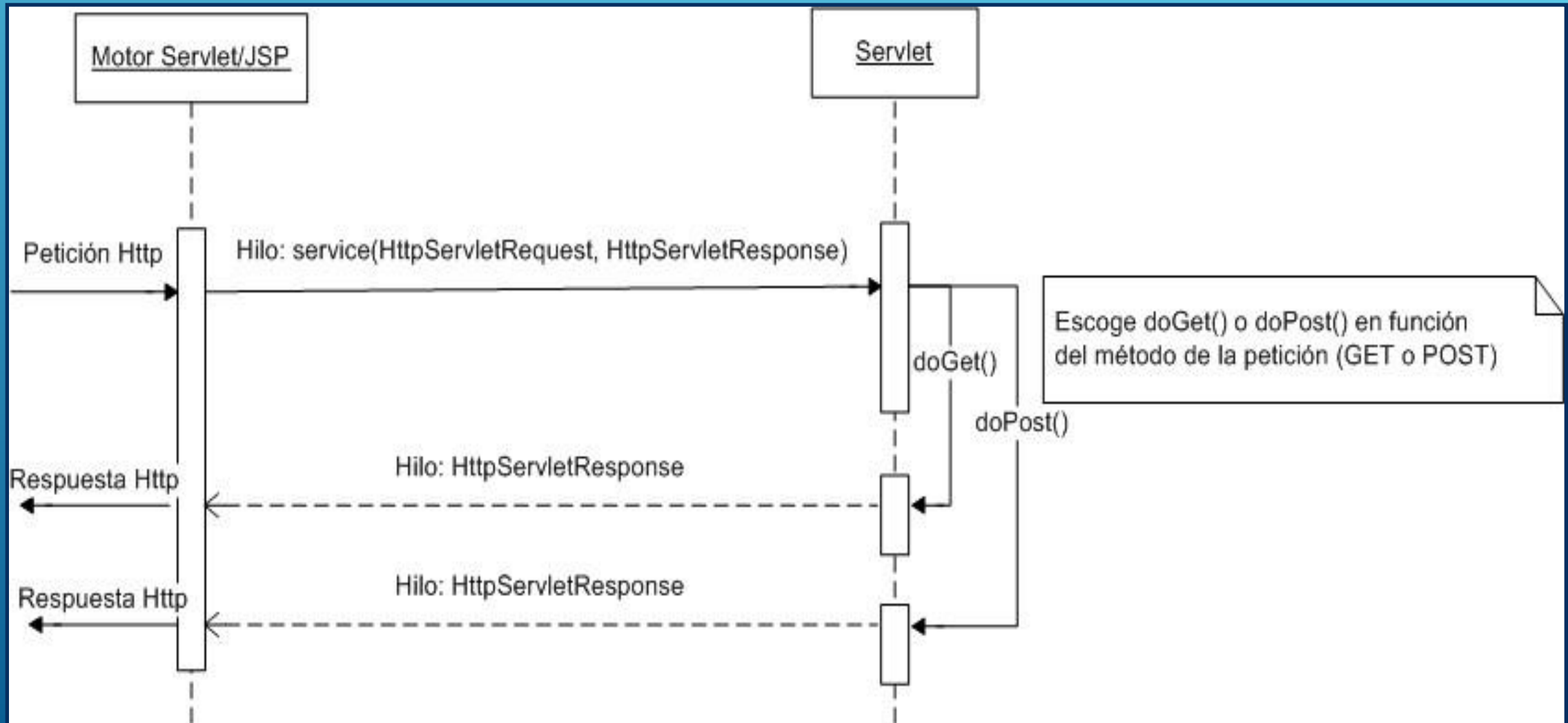
SERVLET

Proceso de ejecución de un Servlet:

- El motor crea por cada petición un hilo, sobre el que se invoca a la función `service()` del servlet. En función del método de la petición, `service()` llamará al método correspondiente del servlet: `doPost()`, `doGet()` u otro método `doXXX()`, pasándoles los objetos de `HttpServletRequest` y `HttpServletResponse`. Existen más métodos HTTP siendo GET y POST son los más habituales.
- Cada clase del tipo servlet, tiene una única instancia, sobre la que corren los diferentes hilos (peticiones). Sólo la primera petición genera una instancia de la clase servlet. Podremos tener N peticiones al servlet, por tanto N hilos, pero únicamente una instancia de la clase (un objeto).

SERVLET

Proceso de ejecución de un Servlet:



SERVLET

Proceso de ejecución de un Servlet – Acceso simultaneo:

- Al generar por cada petición un nuevo hilo (subproceso) se debe de tener cuidado de sincronizar el acceso a los campos y otro datos compartidos, dado que varios hilos podrían estar accediendo a los datos en forma simultanea.
- Para evitar el acceso múltiple se puede hacer que un **servlet** implemente la interfaz **SingleThreadModel**, lo que realiza esto es poner las peticiones en una cola y pasarlas una a la vez a la unica instancia del **servlet**
- **Hay que tener cuidado al utilizar esto ya que afecta al rendimiento (latencia) de un servlet al trabajar con muchos accesos simultáneos.**

SERVLET

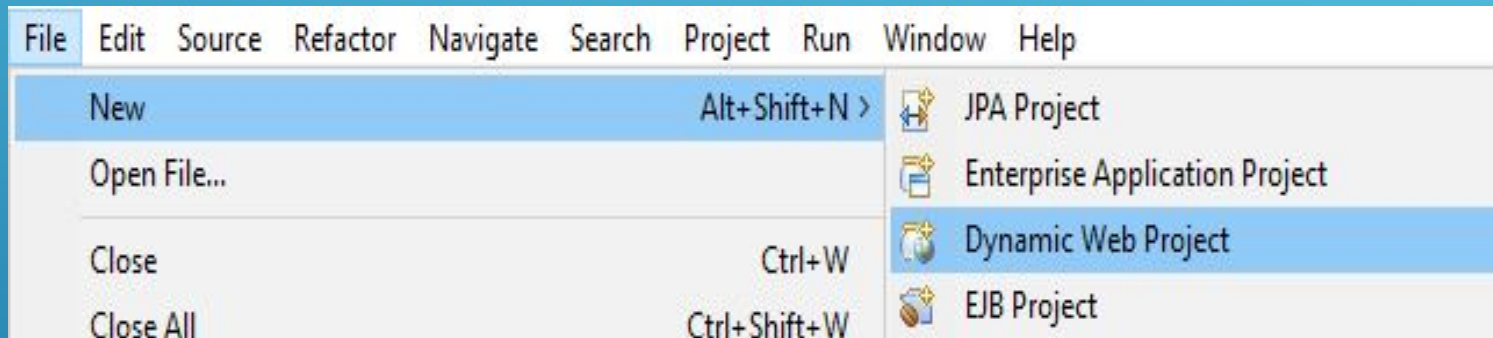
Proceso de ejecución de un Servlet – Destrucción:

- Por algún motivo se decide eliminar la instancia del **servlet**. Antes de hacerlo se ejecuta el método **destroy()** de nuestro **servlet**.
- *Este método nos da la oportunidad de cerrar conexiones a Base de Datos, escribir lista de cookies, acceso al disco, etc.*

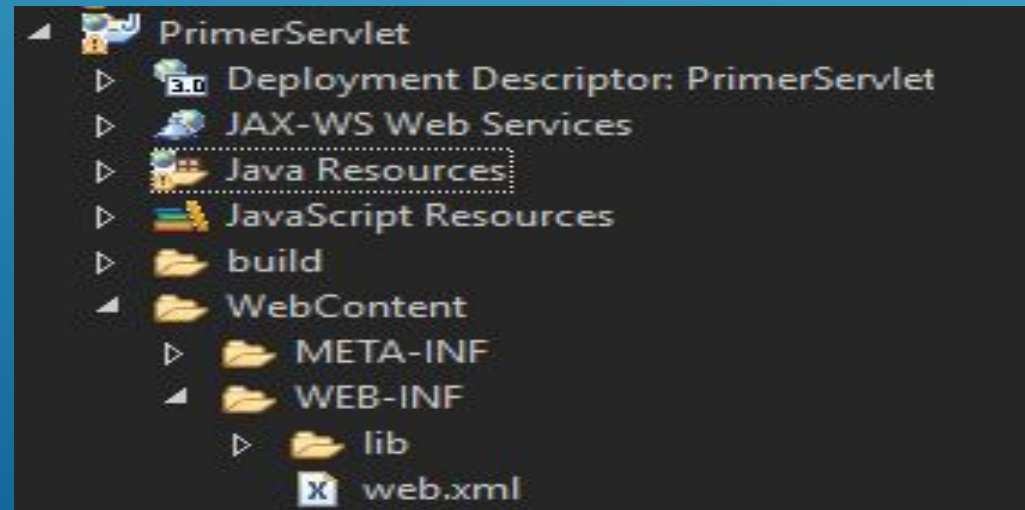
SERVLET

Armado de un Servlet:

Se crea un proyecto Java nuevo: “**Dynamic Web Project**”.



Estructura del proyecto



SERVLET

Armado de un Servlet:

Se crea el **servlet**, que es una clase que extiende de “**HttpServlet**”.

```
public class ServletExample extends HttpServlet {  
    private static final long serialVersionUID = 1L;  
  
    public ServletExample() {  
        super();  
    }  
  
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException {  
        response.getWriter().append("Served at: ").append(request.getContextPath());  
    }  
  
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException {  
        doGet(request, response);  
    }  
}
```

SERVLET

Armado de un Servlet:

Se configura el archivo “**web.xml**”, que inicializa el servlet en el servidor.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns=
  <display-name>PrimerServlet</display-name>
  <servlet>
    <servlet-name>EjemServlet</servlet-name>
    <servlet-class>edu.usal.EjemServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>EjemServlet</servlet-name>
    <url-pattern>/EjemServlet</url-pattern>
  </servlet-mapping>
</web-app>
```

Se genera el exportado, un archivo *.war, que será publicado en el servidor web.

SERVLET

Armado de un Servlet:

Se arma un archivo **.html*, que será el encargado de consumir del servlet, realizando el **request** correspondiente.

```
<H2>Introduce tus datos:</H2><BR>
<HR></HR>
<CENTER>
<FORM NAME="FORM1" METHOD="POST" ACTION="http://localhost:9081/PrimerServlet/EjemServlet">
  <TABLE BORDER>
    <TR>
```

SERVLET

Leer datos del formulario desde el Servlet:

Se pueden obtener los datos del formulario a través del método **`getParameter(String param)`** de nuestro objeto **`HttpServletRequest`**, en el caso de no existir el parámetro nos retornara null, en el caso de existir el valor del parámetro o vacío si no tiene valor.

El método **`getParameterName`** nos retorna una enumeración de los parámetros del formulario.

```
<TR>
  <TD><B>Nombre:</TD>
  <TD><INPUT ENGINE=TEXTBOX NAME="NOM" SIZE="25" VALUE=""></TD>
</TR>
```

```
pw.println("<OL>(1) ",
pw.println("Te llamas " + request.getParameter("NOM") + "<BR>");
pw.println("y tienes " + request.getParameter("EDA") + " años<BR>");
```

SERVLET

HttpServletRequest – Lectura de otros datos.

Además de obtener los datos del formulario, podemos obtener mas información que viaja a través del **request** (petición) realizada al servidor.

Los servlet nos facilitan la lectura de diversos datos, gracias a métodos específicos. Por ejemplo como los siguientes:

QUERY_STRING: con el método **getQueryString()** podemos obtener los datos adjuntos a la petición GET.

REMOTE_ADDR: con el método **getRemoteAddr()** podemos obtener la dirección IP del cliente.

REQUEST_METHOD: con el método **getMethod()** podemos obtener la petición HTTP

SERVLET

HttpServletResponse – Generando respuestas.

De la misma manera que podemos obtener información relevantes desde el **request** realizado, podemos enviar cierto tipo de información para que se interpretada del lado cliente a través de nuestro **response**.

Informar código de estado: es información importante para saber que paso del lado servidor, los códigos de estado suelen estar asociados a algún mensaje. Para enviar esto desde nuestro servlet lo tenemos que hacer a través del método **setStatus(int codigo)**. Los códigos están definidos en constantes de nuestro objeto **HttpServletResponse**.

100-199: propósito informativo, el cliente deberá de responder con otra acción.

200-299: operación exitosa. Por lo general siempre es 200.

300-399: informa si el archivo fue movido informando su nueva ubicación.

400-499: indican un error del cliente.

500-599: indicar un erro del servidor.

SERVLET

HttpServletResponse – Generando respuestas.

Además de informar el código de estado de lo sucedido podemos enviar uno o varios encabezado en nuestra respuesta.

setContentType(String tipo): con este método podemos informar el **Contet-Type** retornado.

sendRedirect(String url): con este método podemos redireccionar a otra pagina es equivalente al codigo de estado 302.

Tipos MIME mas comunes:

- application/pdf – Archivo *.pdf
- application/zip – Archivo *.zip
- text/html – Documento html.
- application/jpeg – Imagen en formato JPEG.
- application/xml – Documento XML.
- application/json – Datos en formato JSon